

HW3 - Seeded Deutsch-Jozsa Oracle Classification

Student Instruction Sheet

This assignment asks you to build and test a personalized Deutsch-Jozsa circuit. Your student ID generates a seeded oracle mask, output offset, and measurement map. You must classify the oracle, build the circuit, run the simulator, interpret the bitstring, and export answers.json.

Learning goals

- Classify Deutsch-Jozsa oracles as constant or balanced.
- Build a custom seeded oracle from a mask.
- Measure only the input register and interpret Qiskit bit ordering.
- Produce machine-readable answers.json for autograding.
- Optionally compare simulator output with IBM hardware output.

Seeded oracle format

Your oracle has the form $f(x) = \text{mask} \cdot x \text{ XOR output_offset}$. The mask is listed in $q[0] \dots q[n-1]$ order. If the mask is all zeros, the oracle is constant. If the mask has at least one 1, the oracle is balanced. The output_offset may flip all outputs but does not change the constant/balanced classification.

Required student tasks

Part	Task	Evidence
A	Generate seeded config	Printed config in notebook
B	Classify oracle before building	predicted_oracle_classification + reflection
C	Implement oracle	X for output_offset and CX gates for mask bits
D	Build full Deutsch-Jozsa circuit	Circuit diagram and simulator execution
E	Predict expected bitstring	expected_logical and expected_display
F	Run simulator	simulator_counts and dominant_bitstring
G	Export artifact	answers.json
H optional	Run IBM hardware	backend, job ID, raw counts, interpretation

Deutsch-Jozsa circuit structure

1. Use n input qubits and 1 output/ancilla qubit.
2. Prepare output in $|1\rangle$, then apply H to make $|-\rangle$.
3. Apply H to every input qubit.
4. Apply the seeded oracle.
5. Apply H to every input qubit again.
6. Measure the input qubits only according to measurement_map.

Bit-ordering warning

Qiskit count strings are displayed in classical-bit order $c[n-1] \dots c[0]$. This can differ from the mask written in $q[0] \dots q[n-1]$ order, especially when the measurement map is reversed. You must use the measurement map to compute the displayed bitstring.

Submission

Submit answers.json. If requested by the instructor, also submit the completed notebook and optional circuit.qpy/hardware evidence.

Instructor notes

This assignment implements the hidden-reference pattern: the grader regenerates the student-specific oracle from the same student ID and assignment ID, then compares answers.json against the computed reference. Exact hardware counts should not be used for strict grading because real hardware is noisy and time-dependent.

AI-resilience layers

- Seeded oracle masks personalize the circuit.

- Students classify before execution, forcing conceptual reasoning.
- Non-palindromic balanced masks expose bit-order reversals.
- Measurement maps vary direct/reversed displayed outputs.
- Simulator execution is required for counts.
- Circuit metrics discourage replacing the algorithm with a shortcut.
- Optional IBM hardware results require backend-specific evidence and noise interpretation.

Suggested grading split

Component	Weight
Oracle classification and explanation	15%
Oracle/circuit construction	25%
Expected bitstring and bit-ordering	20%
Simulator execution and JSON artifact	25%
Reflection/hardware extension	15%